Phase 2: Legal Named Entity Recognition

Six ways to adapt the Phase-1 legal LMs, explained with examples

Manish Kumar (251330604)

CS-781, IIT Kanpur

October 2026

Outline

1. The task
2. From characters to tags and back
3. The adaptation techniques
4. Results
5. Tuning points to try next

L-NER: find the entity, its exact characters, and its legal role

A real training document (fold1, id 115651329)

... JAGE RAM & ORS. ..Appellants Versus
STATE OF HARYANA RESP ..Respondent ... the judgment
dated 19.8.2011 DATE passed by the
High Court of Punjab and Haryana COURT in Criminal ...

Required output (`prediction.json`)

```
{"115651329":  [[137, 153, "RESP"],
                [252, 261, "DATE"],
                [276, 308, "COURT"], ...]}
```

Strict scoring

Start, end *and* label must all match: `[275, 308, "COURT"]` scores zero. Metric: macro-F1 over 12 labels.

Data

- Train: 70 judgments (fold1 + fold2), 8,393 entities.
- Test: 35 judgments (one is empty).
- ≈31k characters ≈6.6k tokens per document.
- No dev split: 10 training docs held out.

Label counts (train)

DATE	1575	APP	407
PREC	1183	RESP	381
STAT	1260	JUDGE	245
COURT	1013	A.COUNSEL	195
AUTH	1003	WIT	187
CASENO	760	R.COUNSEL	184

Macro-F1 weights rare R.COUNSEL equal to DATE: **class imbalance matters**.

Step 1: tokens with character offsets

The Phase-1 byte-level BPE returns, for every token, the characters it covers. A token usually carries its leading space (Ġ), so we strip it:

token	raw offsets	stripped	text
Ġdated	(245, 251)	(246, 251)	dated
Ġ19	(251, 254)	(252, 254)	19
.	(254, 255)	(254, 255)	.
8	(255, 256)	(255, 256)	8
.	(256, 257)	(256, 257)	.
2011	(257, 261)	(257, 261)	2011

Entity `[252, 261, DATE]` = first token start ... last token end.

Problem: punctuation runs are merged

BPE: `(AIR 1990 SC 79).` → `( AIR Ġ1990 ĠSC Ġ79 ).`
The precedent ends after `)` but the token is `).` ⇒ no tagger can output the right end.

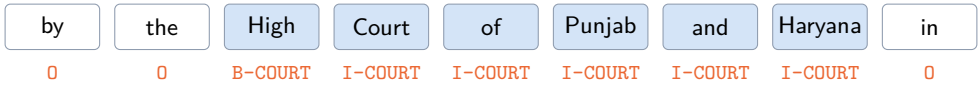
Fix: `data.presplit: true`

Split words / digits / single punctuation *before* BPE: `( AIR Ġ1990 ĠSC Ġ79 ) .` Words stay intact.

	tokens	oracle macro-F1
plain BPE	458,158	0.9866
presplit	465,081	**0.9945**

Oracle: gold → tags → spans, the pipeline's ceiling.

Step 2: BIO tags, one per token (25 classes)



Encoding (training)

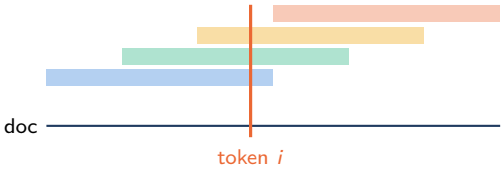
- Tags: `O` + `B-X`, `I-X` for 12 labels = 25.
- A token gets the entity's tag if its visible characters overlap the span.
- First token `B-`, the rest `I-`.

Decoding (prediction)

- Run of `B-X I-X ...` → one span `[start(first), end(last), X]`.
- Trailing whitespace trimmed.
- Alternative design (not used yet): span classification / start–end pointers.

Step 3: long documents, 512-token windows

A document (≈6.6k tokens) is longer than the window, so it is cut into overlapping windows.



Training: window 512, stride 256, a random shift each epoch so entities land at new positions.
Inference: stride 128. Token i is covered by several windows; it keeps the logits of the one where it has the *most context*:

- bidirectional model: $\max \min(\text{left}, \text{right})$ (most central),
- causal model: max left context.

Example

Token 700, stride 128, window 512.
Windows starting at 256, 384, 512, 640 cover it.
Left / right context: 444/67, 316/195, 188/323, 60/451.
Bidirectional pick: start 384 (min = 195).
Causal pick: start 256 (444 tokens on the left).

Why this matters

Parties (APP/RESP) are decided by cues like *"..Appellants Versus"* which can sit in another window. Window size and stride are tuning knobs (later slide).

Step 4: Viterbi decoding keeps the BIO sequence valid

Argmax per token can produce impossible sequences (illustration):

token	`Kashiram`	`&`	`Ors`
argmax	`O`	`I-PREC`	`I-PREC`
Viterbi	`O`	`O`	`O`
or	`B-PREC`	`I-PREC`	`I-PREC`

Viterbi finds the most probable sequence that obeys:

- `I-X` only after `B-X` or `I-X`,
- a document cannot start with `I-X`.

$$\hat{y} = \arg\max_{y \text{ valid}} \sum_{t} \log p(y_t \mid x)$$

Cost: $O(T \cdot 25^2)$, a few ms per document.

Which one does it choose?

Whichever has the higher total log-probability: if `Kashiram` had p(B-PREC) $= 0.3$, O$= 0.6$ and the `I-PREC` tokens are confident, the span version wins.

Strict metric reminder

For every label X: $P_X = \frac{TP}{TP+FP}$, $R_X = \frac{TP}{TP+FN}$, $F1_X = \frac{2P_X R_X}{P_X+R_X}$.
Macro-F1 = mean over the 12 labels.
We also log *span-only F1* (boundaries right, label ignored) to separate boundary errors from label confusion.

Step 5: turning each Phase-1 model into a token encoder

Model	What we feed / read	Attention for tagging
A1 GPT	token + learned positions → 8 blocks → LayerNorm	causal mask removed (`attention: bidirectional`); max 1024 tokens
A2 enc–dec	encoder reads the window; **decoder re-reads it** while cross-attending; tag decoder states (BART-style)	encoder bidirectional, decoder causal + full memory
A3 prefix-LM	whole window = the "prompt" prefix	fully bidirectional, as in Phase-1 prompt reading
A4 efficient	RoPE + GQA; local layers see ±256 tokens, every 3rd layer global	causal mask removed

On top: dropout 0.1 + **linear head** $d \to 25$. Loss: cross-entropy over tags.

A2 finding: the encoder alone is almost useless

Mean cosine between A2 *encoder* token vectors = **0.92** (A3: 0.19): nearly the same vector for every token. Tagging encoder states: 0.11 (adapters), 0.13 (partial FT), 0.15 (full FT) dev macro-F1. Tagging *decoder* states: **0.39**. In Phase 1 the A2 decoder did most of the language modelling.

Overview: what each technique trains

Run	Phase-1 model	Technique	What is trained	Trainable	Checkpoint
E1	A3 prefix-LM	Full fine-tuning	everything	81.30M (100%)	325 MB
E2	A4 efficient	**LoRMA+**	low-rank multiplicative factors	2.23M (2.5%)	9 MB
E3	A1 GPT	LoRA	low-rank additive factors	1.79M (2.1%)	7 MB
E4	A2 enc–dec	Houlsby adapters	bottleneck MLPs + LayerNorms	2.04M (2.2%)	8 MB
E5	A4 efficient	Partial (layer-wise)	top 4 of 10 blocks	25.19M (28.8%)	101 MB
E6	A3 (4 of 8 blocks)	**Distillation**	student, taught by E1	52.95M (100%)	212 MB

- The token-classification head ($\approx$19k params) is trained in every run.
- PEFT checkpoints store only the trained tensors: the Phase-1 weights are re-loaded and the small file is applied on top.
- Also implemented, not submitted: BitFit (`peft.method: bitfit`).

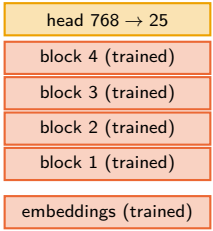
Full fine-tuning (E1, A3 prefix-LM)

Idea. Start from Phase-1 weights W_0 and update every one:

$$W \leftarrow W - \eta \nabla_W \mathcal{L}_{\text{CE}}$$

Example. A3: 8 blocks, $d = 768$, 81.3M parameters, all trainable. Two learning rates: backbone 10^{-4} (small, keeps Phase-1 knowledge), new head 10^{-3} (starts random). Warm-up 10% then cosine decay.

Pros: most capacity; simplest.
Cons: 325 MB copy per task; can overwrite Phase-1 knowledge; most optimizer memory (AdamW: 2 extra values per weight).



What we observed

Train loss → 0.0006; dev macro-F1 stays at 0.39–0.44 from epoch 6: it memorises the 60 training documents.

Partial / layer-wise fine-tuning (E5, A4 efficient)

Idea. Lower layers learn general features (spelling, syntax); upper layers are more task-specific. Freeze the bottom, train the top.

Example. A4 has 10 blocks. `n_train_layers: 4` trains blocks 7–10, the final RMSNorm and the head:

$$\underbrace{4 \times 6.3\text{M}}_{\text{blocks}} + \text{norm} + \text{head} = 25.2\text{M}\ (28.8\%)$$

Embeddings (24.6M) and blocks 1–6 stay frozen.

Pros: no gradients through frozen layers ⇒ faster (96k tok/s vs 43k for E2) and less memory (2.3 GB).
Cons: which layers to train is a guess; frozen features may be suboptimal.



Result: 0.470 dev macro-F1, best epoch 6 (earliest of all runs).

LoRA: low-rank *additive* update (E3, A1 GPT)

Idea (Hu et al., 2022). The weight change needed for a new task has low rank. Keep W_0 frozen and learn $\Delta W = BA$:

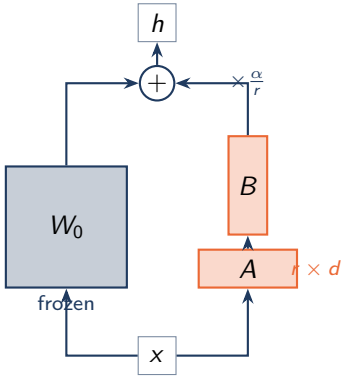
$$h = W_0 x + \frac{\alpha}{r} BAx, \qquad B \in \mathbb{R}^{d_{out} \times r},\ A \in \mathbb{R}^{r \times d_{in}}$$

$B = 0$ at start $\Rightarrow$ the model begins exactly as the Phase-1 model.

Parameter count for one A1 query matrix

W_0: $768 \times 768 = 589{,}824$ (frozen)
$r = 16$: A 16×768 + B 768×16 = **24,576** (4.2%)
Applied to q, k, v, o, fc, proj in 8 blocks: 1.77M + head = 1.79M.

$\alpha = 32$, $r = 16 \Rightarrow$ scale 2. Learning rate 10^{-3} (only small new matrices).



Result: 0.356. Note A1 was trained *causal*; we remove the mask, and LoRA's 2% of parameters must adapt to that change too.

LoRMA: low-rank *multiplicative* update (E2, A4 efficient)

Idea (Bihany, Patel & Modi, 2025). Instead of *adding* a matrix, *multiply* the frozen weight by a learned transformation:

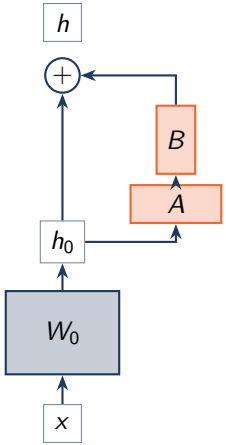
$$W = \left(I + \tfrac{\alpha}{r} BA\right) W_0, \quad B \in \mathbb{R}^{d_{out} \times r},\ A \in \mathbb{R}^{r \times d_{out}}$$

Rank inflation. BAW_0 alone has rank $\leq r$, too restrictive. Adding I (*LoRMA+*) makes the update full rank; with $B = 0$ it starts at W_0.

Reorder for speed (never form BAW_0):

$$h_0 = W_0 x; \quad h = h_0 + \tfrac{\alpha}{r} B(Ah_0)$$

Same cost as LoRA. A acts on the *output* h_0, not on x.



A4 numbers ($r = 16$; params $= 2r\, d_{out}$)

q, o, proj: 24,576 each; k, v (GQA, 256 out): 8,192 each; gate, up (2048 out): 65,536 each. $\times 10$ blocks + head = **2.23M** (2.5%). Dev macro-F1 **0.486** (best run).

LoRA vs LoRMA side by side

	LoRA	LoRMA+
Update	$W = W_0 + \frac{\alpha}{r} BA$	$W = (I + \frac{\alpha}{r} BA) W_0$
A shape	$r \times d_{in}$ (reads the input x)	$r \times d_{out}$ (reads the output $W_0 x$)
Rank of the change	$\leq r$	up to full (BAW_0 mixes all of W_0's directions)
Init	A random, $B = 0$	A random, $B = 0$
Extra compute	$B(Ax)$	$B(A\,W_0 x)$, reusing $W_0 x$
Params per $d \times d$ matrix	$2rd$	$2rd$
Params when $d_{out} \neq d_{in}$	$r(d_{in} + d_{out})$	$2r\,d_{out}$

Toy example, $d = 2$, $r = 1$

$W_0 = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$, $B = \begin{pmatrix} 0.5 \\ 0 \end{pmatrix}$, $A = \begin{pmatrix} 0 & 1 \end{pmatrix}$, $\frac{\alpha}{r} = 1$. LoRA: $W = \begin{pmatrix} 1 & 0.5 \\ 0 & 2 \end{pmatrix}$. LoRMA+:

$$(I + BA) W_0 = \begin{pmatrix} 1 & 0.5 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix}.$$

LoRMA's update is *scaled by what W_0 already does*: it re-combines existing output features.

E2 vs E3 also differ in model (A4 vs A1): run LoRA on A4 to compare methods alone.

Houlsby adapters (E4, A2 encoder–decoder)

Idea (Houlsby et al., 2019). Insert a small bottleneck MLP after a frozen sub-layer:

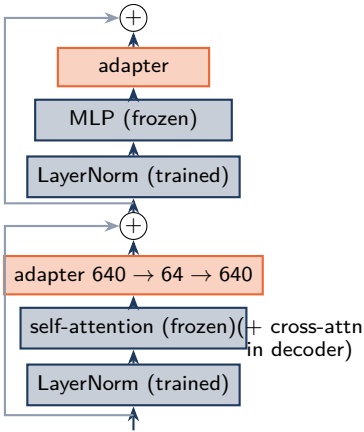
$$y' = y + W_{up}\,\mathrm{GELU}(W_{down}\,y)$$

$W_{down} : d \to m$, $W_{up} : m \to d$, $W_{up} = 0$ at start (identity). Placed after **attention** and after the **MLP** of every block; LayerNorms are also trained.

A2 numbers ($d = 640$, $m = 64$)

One adapter: $640 \cdot 64 + 64 + 64 \cdot 640 + 640 = 82{,}624$
2 per block $\times$ 12 blocks (6 enc + 6 dec) $= 1.98$M
+ 32 LayerNorms (41k) + head (16k) = **2.04M** (2.2%)

vs LoRA: adds sequential layers (slightly slower inference) but can apply a *non-linear* correction.



Result: **0.389** with decoder states (0.112 on encoder states).

BitFit (implemented, extra option)

Idea (Ben Zaken et al., 2022). Train only the bias vectors (and norm gains); every weight matrix stays frozen:

$$h = W_0 x + \mathbf{b}, \qquad \text{only } \mathbf{b} \text{ is trained}$$

Example. A3 block: biases of q,k,v,o (4×768), fc (3072), proj (768), two LayerNorms (4×768) ≈ 10.0k per block; 8 blocks + final norm + head = **100.6k**: **0.12%** of the model.

Caveat. A4 uses bias-free linears and RMSNorm: BitFit on A4 trains only norm gains. Use it on A1, A2 or A3.

```
python train.py -config
configs/e1_a3_full.yaml
  -set peft.method=bitfit train.lr=1e-3
run_name=E7_a3_bitfit
```

Where it sits

method	trainable
BitFit (A3)	0.12%
LoRA / LoRMA / adapters	2–2.5%
Partial (top 4/10)	29%
Full	100%

A useful lower bound: how much can the Phase-1 features do with almost no change?

Knowledge distillation (E6): teacher E1 $\rightarrow$ 4-block A3 student

Idea (Hinton et al., 2015). A small student imitates the full *probability distribution* of a trained teacher, not only the gold tag:

$$\mathcal{L} = \alpha\,\mathrm{CE}(y, p_s) + (1-\alpha)\,T^2\,\mathrm{KL}\big(p_t^{(T)} \,\|\, p_s^{(T)}\big)$$

$p^{(T)} = \mathrm{softmax}(z/T)$. We use $\alpha = 0.5$, $T = 2$. T^2 keeps gradient sizes comparable as T changes.

Student init (DistilBERT-style): keep Phase-1 A3 blocks 2, 4, 6, 8 (`keep_layers: [1,3,5,7]`), embeddings and final norm.

	teacher E1	student E6
blocks	8	4
params	81.3M	52.9M
tokens/s (train)	91k	130k
dev macro-F1	0.439	0.344

Why soften with T? A token's teacher logits

$z = (4, 2, 0)$ for (`B-COURT`, `B-AUTH`, `O`)

$T = 1$: $p = (0.87, 0.12, 0.02)$
$T = 2$: $p = (0.67, 0.24, 0.09)$

At $T = 2$ the student also learns "this looks a bit like an authority": the *dark knowledge* about which labels are confusable.

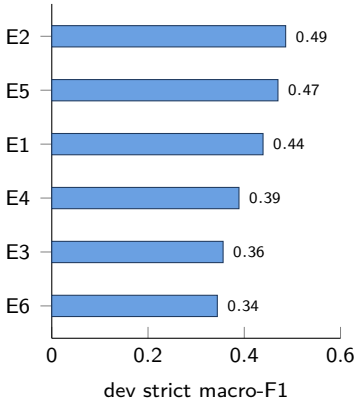
Params shrink less than blocks

Halving the blocks removes 28M, but the 24.6M embedding table stays. Smaller vocab / d would compress further.

Dev results (10 held-out documents, strict)

Run	macro-F1	P	R	span-only	epoch
E1 A3 full	0.439	0.457	0.453	0.566	13
E2 A4 LoRMA	**0.486**	0.518	0.478	0.573	13
E3 A1 LoRA	0.356	0.395	0.356	0.503	15
E4 A2 adapter	0.389	0.422	0.389	0.489	14
E5 A4 partial	0.470	0.488	0.474	0.559	6
E6 A3 KD	0.344	0.372	0.344	0.483	9

Run	train min	peak GPU	tok/s
E1	2.1	3.7 GB	91k
E2	4.4	4.5 GB	43k
E3	2.8	3.2 GB	67k
E4	3.1	2.9 GB	62k
E5	1.4	2.3 GB	96k
E6	1.4	2.6 GB	130k



Span-only F1 (boundary right, any label) is 0.08–0.13 above macro-F1: both boundaries *and* labels lose points.

Per-label dev F1: where the points are lost

Run	APP	RESP	A.CO	R.CO	JUDGE	WIT	AUTH	COURT	STAT	PREC	DATE	CASENO
E1	.12	.05	.32	.40	.45	.29	.19	.73	.70	.44	.98	.60
E2	.12	.06	.62	.67	.47	.21	.20	.79	.67	.40	.98	.64
E3	.02	.00	.08	.26	.39	.18	.18	.74	.64	.31	.98	.49
E4	.06	.11	.42	.48	.27	.17	.16	.76	.60	.28	.95	.42
E5	.05	.00	.50	.48	.51	.43	.25	.79	.67	.46	.96	.54
E6	.08	.00	.17	.18	.41	.21	.22	.74	.66	.23	.92	.32
dev gold	114	93	20	31	33	44	226	97	165	232	190	60

Example of an APP error (E1, test doc)

Text: `...HOUSING CO-OPEATIVE SOCIETY LTD.` E1 predicted `PAN` / `OPEATIVE` / `SOCIETY LTD`: three APP fragments. ALL-CAPS splits oddly: `ESSCO FABS` → `ES SCO ĠF AB S`.

Questions for the analysis

- APP/RESP vs AUTH/COURT: label confusion or boundaries?
- How many errors sit in the document header?
- 20–44 gold A.COUNSEL / WIT: stable across seeds?

Tuning points (1/3): data, context, imbalance

Knob	Why it may help	How to run
Train on all 70 docs	+17% data; keep the epoch count found on dev	`train.py ... -all-data -set train.epochs=13`
Cross-validation on folds	10 dev docs are noisy (±0.03); fold1↔fold2 gives 35-doc estimates	`-set data.split_seed=1` (and a fold-split option)
Longer window	party cues ("..Appellants Versus") further away; A3/A4 have no length limit	`-set train.window=1024 infer.window=1024 train.batch_size=8`
Smaller inference stride	every token gets a more central window	`predict.py ... -set infer.stride=64`
Down-weight O	91% of tokens are O; rare labels are recall-limited	`-set train.o_weight=0.5`
Per-label weights / focal loss	macro-F1 counts R.COUNSEL as much as DATE	new loss option: $w_X \propto 1/\sqrt{n_X}$ or $(1-p)^\gamma$ CE
Case-normalise ALL-CAPS lines	`ESSCO FABS` → `Essco Fabs` tokenizes into familiar pieces; offsets unchanged (same length for ASCII)	preprocessing in `lner/data.py` before `tokenize`

Tuning points (2/3): heads, decoding, post-processing

Knob	Why it may help	How to run
MLP head	non-linear head over frozen PEFT features	`-set model.head=mlp`
CRF layer	learns transition scores instead of hard Viterbi rules; a common gain in NER	new head: emissions + learned 25×25 transitions
Span-based head	classify (start, end) pairs directly; fixes split entities like `PAN / OPEATIVE`	biaffine scorer over token pairs, max span length $\approx$40 tokens
Argmax vs Viterbi	measure what the constraints buy	`predict.py -split dev -set infer.decode=argmax`
Merge adjacent same-label spans	`SOCIETY` + `LTD` split by one token	post-process in `tags_to_spans`
Document consistency	a party named once in the header recurs later with the same label	propagate high-confidence strings across the document
Boundary trimming rules	trailing `.`, `,` or `)` wrongly included/excluded	compare predicted vs gold ends on dev, add rules

Tuning points (3/3): adaptation and distillation

Knob	Why it may help	How to run
Same method, other model	separates method effect from model effect (LoRA on A4, LoRMA on A1/A3)	e3 config `-set model.phase1_ckpt=runs/A4_efficient/best.pt`
LoRA/LoRMA targets and rank	attention-only vs +MLP; $r \in \{4, 8, 32\}$	`-set peft.targets=[q,v] peft.r=8`
Causal vs bidirectional	does removing A1/A4's causal mask help or hurt?	`-set model.attention=causal`
Layer-wise LR decay	full FT: smaller LR for lower layers protects Phase-1 features	optimizer groups per block, $\eta_l = \eta\,\xi^{L-l}$
Regularisation	E1 reaches loss 0.0006: overfitting	`-set model.dropout=0.2 train.weight_decay=0.1`
Fair KD baseline	same 4-block student *without* the teacher shows the KD gain	`e6 config -set distill=null run_name=E6_noKD`
Better teacher / KD knobs	E2 is stronger than E1; sweep α, T; add hidden-state matching	`-set distill.alpha=0.3 distill.temperature=4`
Further PEFT methods	extra submissions: prefix / prompt tuning, IA^3, $LoRMA_\pi$	new `peft.method` entries

Reproduce everything

Commands (from p2/)

`bash run_all.sh`	alignment check, E1–E6, six zips in `submissions/`
`python train.py -config configs/e2_a4_lorma.yaml`	one experiment
`python predict.py -run runs/E2_a4_lorma`	test zip
`python predict.py -run runs/E2_a4_lorma -split dev`	per-label dev report
`python scripts/compare.py`	`runs/comparison.md`

Code map

`lner/data.py`	offsets, presplit, BIO encoding/decoding, windows
`lner/backbones.py`	A1–A4 as token encoders, tagging head, KD student layers
`lner/peft.py`	full, partial, BitFit, LoRA, LoRMA+, Houlsby adapters
`lner/decoding.py`	argmax / Viterbi
`lner/engine.py`	sliding-window inference, run loading
`lner/metrics.py`	strict macro P/R/F1, span-only F1

References

- H. Bihany, S. Patel, A. Modi. *LoRMA: Low-Rank Multiplicative Adaptation for LLMs*. Findings of ACL 2025. arXiv:2506.07621.
- E. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR 2022.
- N. Houlsby et al. *Parameter-Efficient Transfer Learning for NLP*. ICML 2019.
- E. Ben Zaken, S. Ravfogel, Y. Goldberg. *BitFit*. ACL 2022.
- G. Hinton, O. Vinyals, J. Dean. *Distilling the Knowledge in a Neural Network*. 2015.
- V. Sanh et al. *DistilBERT*. 2019.
- M. Lewis et al. *BART*. ACL 2020 (decoder states for token classification).
- L. Ramshaw, M. Marcus. *Text Chunking using Transformation-Based Learning* (BIO tags). 1995.
- J. Lafferty et al. *Conditional Random Fields*. ICML 2001.